LocalCloudConfigurator

System Design Description

Abstract
This document describes the implementation of the LocalCloudConfigurator support system of the Eclipse Arrowhead framework (documentation edition 5.3.0) in `ah5-LocalCloudConfigurator-java` of the primary Java source tree (`Eclipse-Arrowhead-java`).

Contents

1 Documentation edition and implementation scope **3**

2 Overview **3**

3 Implementation **4**
3.1 Implementation language and tools 4
3.2 Functional properties implementation 4
3.3 Non functional properties implementation 6
3.4 Deployment 6
3.5 Verification 6

4 Services **7**

5 Go and Rust implementation variants **7**
5.1 Go implementation 7
5.2 Rust implementation 7

6 References **7**

7 Revision History **9**
7.1 Amendments 9
7.2 Quality Assurance 9

1 Documentation edition and implementation scope

This is the 5.3.0 documentation edition, dated 2026-11-09. It carries forward the system and service contracts from the retained 5.2.2 documentation and identifies the current primary implementation sources. The former `documentation/v5.2.2` folder is retained unchanged.

Document version 5.3.0 is distinct from executable, dependency, registered system/service and wire-contract versions. Values still defined by the source (including 5.2.2 artefact versions, 1.0.0 service versions and `SysMonitor-AMQP-IDD-5.2.2` payload identifiers) are preserved. This documentation update does not change runtime behaviour or establish a new implementation release. Historical verification statements describe the tests recorded in the inherited baseline; this revision validates document compilation and references.

2 Overview

This document describes the LocalCloudConfigurator system implementation, which realizes the LocalCloud-Configurator SysD [1]. In Section 3, we describe implementation details of the system.

3 Implementation

The implementation is located in `ah5-LocalCloudConfigurator-java` (artifact `arrowhead-localcloudconfigurator`, `localcloudconfigurator.jar`). The same JAR runs as the Arrowhead system (`serve`, default), as the host worker (`worker`) and as the retained command-line tools (`keygen`, `register`, `executor`, `plan`).

3.1 Implementation language and tools

- Java 21, Spring Boot 3.4 (Web, Security, Data JPA, Quartz), Log4j2 with the JDBC log table, MariaDB driver, BouncyCastle (ColonyOS signatures).
- Eclipse Arrowhead common library as a private vendored source copy (`vendor/`).
- Storage: MySQL 8.4 (private schema `ah_localcloudconfigurator`); the host worker uses files.
- Host worker: non-web Spring application with the JDK `HttpsServer`.

3.2 Functional properties implementation

Package `eu.arrowhead.localcloudconfigurator`:

Component	Responsibility
`LocalCloudConfigurator Main`	Command dispatch: Arrowhead system, `worker` (non-web application), tools.
`LocalCloudConfigurator SystemInfo`	System `LocalCloudConfigurator`; services `localCloud Configuration` and `generalManagement`.
`Catalog`	Loads and validates the catalog (names, ownership of databases, scripts, dependency cycles, defaults); dependency closure, protected components, catalog hash.
`Planner`	Plans: defaults and selection with dependencies in order, provision, start, policy, explicit stops in reverse order, optional verification; plan hash.
`Runner`	Runs one step after checking it against the catalog (owner, kind, script, catalog hash, protected components, running dependents, database order); `docker compose` argument lists; output bounded and redacted.
`DirectBackend`	Worker execution: one deployment at a time on a single thread, results written atomically after every step, state-directory lock, interrupted deployments marked failed at start-up and never replayed.
`CloudColony`, `colony/`	Legacy ColonyOS backend: signed RPC, chained `lcc.step` workflows, `lcc.status`.
`service/Deployment Coordinator`	Status, plan and serialized apply with the plan-hash check and the active-deployment guard; shared by the API (JPA store) and the worker (file store).
`service/Configuration Service`	Saved designs with validation and revision checks.
`service/WorkerClient`, `tls/MutualTls`	API side of the worker connection: mutual TLS with peer-name pinning and the CRL.

Component	Responsibility
`service/BackendConfig`, `ColonyBackend`	Catalog bean and backend selection (`lcc.backend`).
`jpa/`	Entities `SavedConfiguration` (`configuration`), `Deployment`, `ActiveDeployment`, `Logs`; `JpaDeploymentStore`.
`api/http/`	`LocalCloudConfigurationAPI` (strict JSON, 64 KiB, native status codes and error bodies), `GeneralManagementAPI`, response headers.
`security/`	`ConfiguratorAccessFilter` and `ConfiguratorAuthorizationService`: Sysop, whitelist or ConsumerAuthorization grant per operation.
`worker/`	`WorkerApplication` (beans of the worker) and `WorkerServer` (loopback HTTPS, client authentication required).

3.2.1 Database

Table	Columns
`configuration`	`id` (UUID), `name`, `systems`, `removals`, `layout` (JSON text), `reapply`, `smoke`, `revision`, `updated_at`.
`deployment`	`id`, `plan`, `request` (JSON text), `state`, `result` (last reported state and steps), `submitted_at`, `updated_at`.
`active_deployment`	At most one row (`id` 1): `deployment_id` of the deployment that blocks new ones until the backend reports `success` or `failed`.
`logs`	Log4j2 JDBC appender (`generalManagement get-log`).

`deployment/database/init.sql` creates the schema; `least-privilege.sh` restricts the application account to SELECT, INSERT, UPDATE and DELETE. Hibernate validates the schema at start-up.

3.2.2 Host worker

The worker is started as `java -jar localcloudconfigurator.jar worker --lcc...` by a per-user launchd service installed with `scripts/install-direct.py` (RunAtLoad, KeepAlive). It does not register in the ServiceRegistry; it is the configurator's local agent. Properties:

Property	Meaning
`lcc.worker.address`, `lcc.worker.port`	127.0.0.1, 8493; a non-loopback address is refused at start-up.
`lcc.worker.ssl.key-store`, `trust-store`, `*-password`	Worker certificate `LocalCloudConfiguratorWorker` (PKCS12), cloud truststore; passwords from a 0600 config tree.
`lcc.worker.ssl.crl-file`	Cloud CRL for client certificates (optional, recommended).
`lcc.worker.allowed-clients`	`LocalCloudConfigurator`.
`lcc.worker.cloud-dir`, `lcc.worker.docker`	Cloud view (`compose.yaml`, `scripts`, `.env`), Docker CLI.

Property	Meaning
`lcc.worker.state`, `lcc.catalog.file`	File state (`active-deployment.json`, `deployments/`, `workflows/`), the worker's own catalog copy.

Worker interface (HTTPS, client certificate required, JSON as in the IDD [2]):

Method	Path	Behaviour
GET	`/healthz`	`{"status": "ok", "executionBackend": "direct"}`.
GET	`/api/status`	Running Compose services and component states.
POST	`/api/plan`	Plan for a request against the worker's catalog.
POST	`/api/deployments`	Reviewed selection with `expectedPlanHash`; the worker rebuilds the plan and refuses a different hash (409); 202 with a `local-<uuid>` identifier.
GET	`/api/deployments/{id}`	State and step results of a worker deployment.

3.3 Non functional properties implementation

3.3.1 Security

Mutual TLS on 8492 and to the worker (SysD, Section Security); outsourced authentication and the access filter for every operation; security profile guard; DML-only database account; read-only root filesystem.

3.3.2 Power management

None. The `sysMonitor` `power` observation always reports line supply.

3.3.3 Internal monitoring

Logs to the console, a rolling file and the `logs` table; the worker logs to the console (launchd log files). sysMonitor observations every 30 s.

3.3.4 Configuration

Spring properties (SysD), secrets from `configtree:/run/secrets/`.

3.4 Deployment

`compose.yaml` (project `ah5-localcloudconfigurator`) runs `localcloudconfigurator` (image `arrowhead-localcloudconfigurator-standalone:5.2.2`, UID 10001, read-only root filesystem, port 8492 on the host's loopback by default, `host.docker.internal` mapped to the host) and `localcloudconfigurator-db` (MySQL 8.4.11 with TLS, internal network). `scripts/deploy.sh` puts both services into a cloud's Compose file and installs the worker; it changes a deployed cloud and is run only by an operator.

3.5 Verification

`mvn clean verify`: catalog, planner and runner tests; direct backend (persistence, no replay, single worker); deployment coordinator (guard across restarts, stale hash); a full-application MockMvc test of every `/api` route with identity and grant checks, size limit and strict JSON; a plan-to-deployment run against a worker over mutual TLS and refusal of a missing, foreign, wrongly named or revoked certificate; the security profile test. A gated test validates the schema against a fresh MySQL 8.4.11 container and checks the DML-only grants.

4 Services

Services produced	SysD ref	SD ref	IDD ref
`localCloudConfiguration`	[1]	[3]	[2]
`sysMonitor`	[1]	[4]	[5]
`generalManagement`	[1]	–	–
Services consumed	**SysD ref**	**SD ref**	**IDD ref**
systemDiscovery, serviceDiscovery	[6]	[7]	–
identity	[8]	[9]	–
authorization (verify)	[10]	–	–

5 Go and Rust implementation variants

5.1 Go implementation

Primary system folder: `Eclipse-Arrowhead-go/ah5-LocalCloudConfigurator-go`.

The system owns its Go module, launcher and build configuration. Use the per-system `Makefile` and `go.mod`; the containing Go tree is the build source, not a historical Arrowhead copy. This is a native operator API/UI using `net/http` and ColonyOS. The `/api` routes require an operator bearer token, not an Arrowhead identity token. It is a distinct binding from the Java support-system wrapper. `Eclipse-Arrowhead-go/ah5-LocalCloudConfigurator-go/internal/api/api.go` records the handlers.

Listening port. serve listens on 127.0.0.1:8095 by default. Override with -listen host:port, SERVER_PORT=N or -server.port N; the port overrides keep the -listen host. For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

5.2 Rust implementation

No Rust implementation folder for LocalCloudConfigurator is present in the designated primary Rust tree. There is no port-specific executable, service binding or parity claim in this edition. A future port needs its own build, state model, security and interface qualification.

6 References

[1] J. Delsing, “LocalCloudConfigurator – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, ah5-LocalCloudConfigurator-java/documentation/v5.3.0/LocalCloudConfigurator-v5.3.0-SysD.pdf.

[2] ——, “localCloudConfiguration HTTP/TLS/JSON – Interface Design Description (LocalCloudConfigurator),” Eclipse Arrowhead, IDD 5.3.0, 2026, localCloudConfigurator-v5.3.0-localCloudConfiguration-HTTP-IDD.pdf.

[3] ——, “localCloudConfiguration – Service Description (LocalCloudConfigurator),” Eclipse Arrowhead, SD 5.3.0, 2026, localCloudConfigurator-v5.3.0-localCloudConfiguration-SD.pdf.

[4] ——, “sysMonitor – Service Description (LocalCloudConfigurator),” Eclipse Arrowhead, SD 5.3.0, 2026, localCloudConfigurator-v5.3.0-sysMonitor-SD.pdf.

[5] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (LocalCloudConfigurator),” Eclipse Arrowhead, IDD 5.3.0, 2026, localCloudConfigurator-v5.3.0-sysMonitor-AMQP-IDD.pdf.

[6] ——, “ServiceRegistry – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.3.0/ServiceRegistry-v5.3.0-SysD.pdf.

[7] ——, “serviceDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.3.0, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.3.0/ServiceRegistry-v5.3.0-serviceDiscovery-SD.pdf.

[8] ——, “Authentication – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, ah5-core-java-spring/authentication/documentation/v5.3.0/Authentication-v5.3.0-SysD.pdf.

[9] ——, “identity – Service Description (Authentication),” Eclipse Arrowhead, SD 5.3.0, 2026, ah5-core-java-spring/authentication/documentation/v5.3.0/Authentication-v5.3.0-identity-SD.pdf.

[10] ——, “ConsumerAuthorization – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, ah5-core-java-spring/consumer-authorization/documentation/v5.3.0/ConsumerAuthorization-v5.3.0-SysD.pdf.

7 Revision History

7.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-11-09	5.2.2	Document created for the Java-Spring implementation: components, database, host worker (non-web Spring application, loopback 8493, mutual TLS, not registered), deployment and verification.	Claude Opus 5.5 (draft)
2	2026-11-09	5.3.0	New documentation edition; primary-source references and Go/Rust implementation scope; 5.2.2 retained.	Documentation maintenance

7.2 Quality Assurance

No.	Date	Version	Approved by
1		5.3.0	